

STM32 I2C Peripheral

Ankit Srivastava

EE2028

STM32L4S5xx Datasheet

STM32L4+ User Manual RM0432

B-L4S5I-IOT01A User Manual UM2708

HTS221 Datasheet

Acknowledgement: Slides adapted from Dr Rajesh Panicker

OVERVIEW

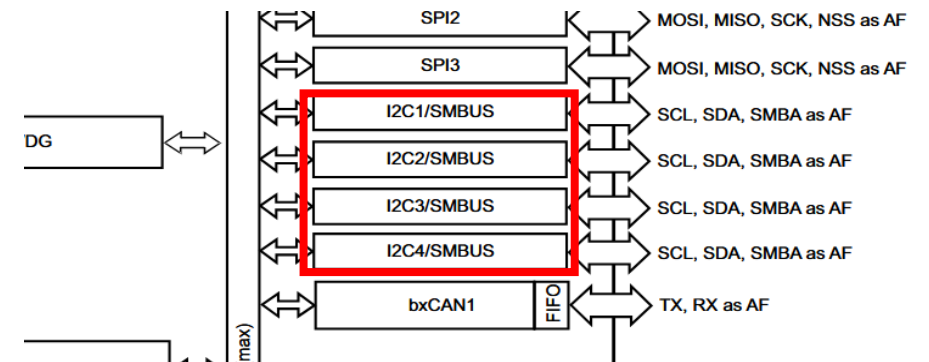
- STM32L4+ I2C interface
 - STM32L4S5VI as an example
- STM32L4+ I2C peripheral registers
 - Control
 - Data
 - Status

I2C IN STM32L4+

- STM32L4S5VI chip has 4 I2C bus interfaces handling communications between the microcontroller and the serial I2C bus
 - controls all I2C bus-specific sequencing, protocol, arbitration and timing
 - converts data from serial to parallel format and vice versa

- The I2C peripheral supports

- Slave and master modes
- Multi-master capability
- Standard-mode, Fast-mode, Fast-mode Plus
- 7-bit and 10-bit addressing mode
- In slave mode - multiple 7-bit slave addresses and optional clock stretching



- Self-reading
 - Which I2C interface is used for interfacing the devices on the Board?
 - Which pins are used for SDA and SCL?
 - Which MODE? Which AF? What PUPD and OTYPE values should be used?

CONTROL REGISTER I2C_CR1 (BASIC CTRL / ENABLES)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PECEN	ALERT EN	SMBD EN	SMBH EN	GCEN	WUPE N	NOSTR ETCH	SBC
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RXDMA EN	TXDMA EN	Res.	ANF OFF	DNF[3:0]				ERRIE	TCIE	STOP IE	NACK IE	ADDR IE	RXIE	TXIE	PE
rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Address offset: 0x00,
Default value: 0x0000 0000

- xxIE is to enable various I2C interrupts. We use polling, so it will all be 0
- Writing a 0 to PE disables and performs a soft reset of the I2C interface
 - SCL and SDA are released. Internal states machines are reset
 - Some communication control bits (in I2C_CR2, e.g.: START, STOP) as well as status bits (in I2C_ISR, e.g: TXIS, RXNE) go back to their Default value. Other I2C_CR1 bits are unaffected
- Writing a 1 enables the I2C interface. Before doing this,
 - Set analog / digital noise filters
 - Configure I2C master clock by setting the SCLH and SCLL bits in the I2C_TIMINGR (which sets speed modes etc.)
 - Set an appropriate value for NOSTRETCH (useful only in slave mode)

© Copyright National University of Singapore. All Rights Reserved.

SENSOR IO INIT

```
/*stm32l4s5i_iot01_tsensor.c*/
void SENSOR_IO_Init(void)
{
    I2Cx_Init(&hI2cHandler);
}

static void I2Cx_Init(I2C_HandleTypeDef *i2c_handler)
{
    /* I2C configuration */
    i2c_handler->Instance          = DISCOVERY_I2Cx;
    i2c_handler->Init.Timing        = DISCOVERY_I2Cx_TIMING;
    i2c_handler->Init.OwnAddress1    = 0;
    i2c_handler->Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    i2c_handler->Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    i2c_handler->Init.OwnAddress2    = 0;
    i2c_handler->Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    i2c_handler->Init.NoStretchMode  = I2C_NOSTRETCH_DISABLE;

    /* Init the I2C */
    I2Cx_MspInit(i2c_handler);
    HAL_I2C_Init(i2c_handler);

    /**Configure Analogue filter */
    HAL_I2CEx_ConfigAnalogFilter(i2c_handler, I2C_ANALOGFILTER_ENABLE);
}
```

Note : A function with static qualifier is a 'private' function, meant to be used by other functions within the file, rather than directly from the user (main) program. Those which are called from the main program are 'public' functions

DATA REGISTERS I2C_RXDR & I2C_TXDR

Receive data register (I2C_RXDR)

- Address offset: 0x24, Default value: 0x0000 0000
- Bits 31:8 Reserved, must be kept at Default value
- Bits 7:0 RXDATA[7:0] → 8-bit receive data : Data byte received from the I2C bus
 - The content from the SIPO (serial-in, parallel-out) shift register which samples the serial data on SDA in receive mode is copied into this register once a complete byte is received

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RXDATA[7:0]							
								r	r	r	r	r	r	r	r

Transmit data register (I2C_TXDR)

- Address offset: 0x28, Default value: 0x0000 0000
- Bits 31:8 Reserved, must be kept at Default value
- Bits 7:0 TXDATA[7:0] → 8-bit transmit data : Data byte to be transmitted to I2C bus
 - The content written to this register is copied into a PISO (parallel-in, serial-out) shift register which sends the data serially on SDA in transmit mode

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TXDATA[7:0]							
								r	r	r	r	r	r	r	r

Note: Every on-chip peripheral register, including I2C_RXDR and I2C_TXDR, is read or written by the processor (i.e., by the software) through parallel interfaces

CONTROL REGISTER I2C_CR2 (COMMUNICATION CONTROL)

Address offset: 0x04,

Default value: 0x0000 0000

read or set. Don't clear
(self-clearing)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	PEC BYTE	AUTOE ND	RE LOAD	NBYTES[7:0]							
					rs	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NACK	STOP	START	HEAD1 OR	ADD10	RD_ WRN	SADD[9:0]									
rs	rs	rs	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- Software (driver) writes to this register to specify information related to one transfer in master mode
- Setting START=1 will cause the interface to enter the master mode (by default, interface is in slave mode) and send a S condition over the bus if the bus is free
 - cleared by hardware after the S followed by the address sequence is sent, by an arbitration loss, by a timeout error detection, or when PE = 0. Writing '0' to this bit has no effect
- Setting STOP will cause P to be generated after current byte transfer
 - cleared by hardware when a P condition is detected (i.e., when P has been sent successfully by the hardware), or when PE = 0. Writing '0' to this bit has no effect
- The I2C interface has a byte counter to generate A / A', P (optional) as appropriate to manage byte transfer
 - while receiving n bytes (n is set in NBYTES) of data, the first n-1 bytes will be A-ed, the last one will be A' ed automatically. P will also be generated if AUTOEND bit is set

CONTROL REGISTER I2C_CR2 (COMMUNICATION CONTROL)

- SADD[9:8] and SADD[0] are don't cares in 7-bit addressing modes, 7-bit slave address should be written to SADD[7:1]
- RD_WRN: transfer direction : 0 → write, 1 → read
- ADD10: 1 → 10-bit addressing mode, 0 → 7-bit mode
- RELOAD: NBYTES reload mode
 - 0 → transfer is completed after the NBYTES data transfer (P / Sr will follow)
 - 1 → transfer is not completed after the NBYTES data transfer (NBYTES will need to be reloaded when TCR* is set). Possible use : when more than 255 bytes are transferred
- AUTOEND: Automatic end mode
 - 0 → software end mode: TC* is set when NBYTES data are transferred, stretching SCL low. The software needs to write to STOP / START to send a P / Sr explicitly
 - 1 → automatic end mode: a P condition is automatically sent when NBYTES data are transferred. Should not be used when we plan to send Sr
 - Ignored when RELOAD is set, as there is still data left to be sent/received

*bit in I2C_ISR

CONTROL REGISTER I2C_CR2 (COMMUNICATION CONTROL)

- NACK (is a control bit) has no effect in master mode
 - A / A' is automatically generated when receiving, not by setting/clearing ACK explicitly
 - When transmitting, we can't read NACK bit to see if the slave has sent a A' by reading this bit – we do that by reading NACKF (is a status bit) of I2C_ISR
- NBYTES, ADD10, RD_WRN, SADD cannot change when the START bit is set

INTERRUPT AND STATUS REGISTER I2C_ISR

Address offset: 0x18,

Default value: 0x0000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADDCODE[6:0]						DIR	
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BUSY	Res.	ALERT	TIME OUT	PEC ERR	OVR	ARLO	BERR	TCR	TC	STOPF	NACKF	ADDR	RXNE	TXIS	TXE
r		r	r	r	r	r	r	r	r	r	r	r	r	rs	rs

- Contains various status and interrupt status bits
 - Generally set by the hardware, cleared due to some action by the software
 - If a bit in this register becomes 1, and the corresponding interrupt is enabled in I2C_CR1, the interrupt is sent to NVIC, and the interrupt handler will be triggered if the interrupt is enabled in the NVIC as well
 - There are 2 separate interrupts from each I2C interface to NVIC : event (EV) and error (ER)
 - Note that TC and TCR interrupts can only be enabled or disabled together in I2C_CR1. Similarly, all error interrupts are enabled or disabled together
 - If not using interrupts, these bits can be polled by the s/w as status bits
 - Clearing some bits (e.g: STOPF) require s/w to write 1 to the corresponding bits (e.g: STOPCF) in I2C_ICR. I2C_ICR has the same bit definition* as I2C_ISR
 - Bits are changed to their default values when PE=0

*For bits available in I2C_ICR

INTERRUPT AND STATUS REGISTER I2C_ISR

- TCR: Transfer Complete Reload
 - Set when RELOAD=1 and NBYTES data have been transferred
 - Cleared when s/w writes a non-zero value to NBYTES
- TC: Transfer Complete (master mode)
 - Set when RELOAD=0, AUTOEND=0 and NBYTES data have been transferred completely
 - Cleared when s/w sets START bit or STOP bit
- RXNE: Receive data register not empty
 - Set when the received data is ready to be read from I2C_RXDR
 - Cleared when I2C_RXDR is read by s/w, following which h/w acknowledges the byte
 - Remains set between the 8th and 9th SCL pulse (before sending ACK)

INTERRUPT AND STATUS REGISTER I2C_ISR

- TXIS: Transmit interrupt status
 - Set when the I2C_TXDR register is empty and the data to be transmitted must be written in the I2C_TXDR register, after the 9th SCL pulse when an ACK is received and NBYTES data have not been transmitted completely
 - Cleared when the next data to be sent is written by s/w to I2C_TXDR
 - Can be written to '1' by s/w in order to generate a TXIS event
- TXE: Transmit data register empty
 - Set when the I2C_TXDR register is empty
 - Cleared when the next data to be sent is written by s/w to I2C_TXDR
 - Can be written to '1' by s/w in order to flush the I2C_TXDR register

TXIS is not set during a read operation even when TXDR is empty, while TXE remains set (TXE=TXDR can be written)

INTERRUPT AND STATUS REGISTER I2C_ISR

- STOPF: Set when a stop condition has appeared on the bus
 - Cleared by writing to STOPCF in I2C_ICR
- NACKF: Set when a byte sent is not acknowledged by the receiver (slave)
 - Cleared by writing to NACKCF in I2C_ICR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ALERT CF	TIMOU TCF	PECCF	OVRCF	ARLOC F	BERRC F	Res.	Res.	STOPC F	NACKC F	ADDR CF	Res.	Res.	Res.
		w	w	w	w	w	w			w	w	w			

I2C_ICR: Interrupt Clear Register

CONCLUSION

- STM32L4+ I2C interface
 - STM32L4S5VI as an example
- STM32L4+ I2C peripheral registers
 - Control – I2C_CR1, I2C_CR2
 - Data – I2C_RXDR, I2C_TXDR
 - Status – I2C_ISR